

Sensor Query Agent

Single-Agent ADK Data Query System

KavAI Home Assignment

Fatemeh Doudi

May 10, 2026

Contents

1	Agent Architecture	2
2	Tool Design	2
3	Example Queries and Outputs	2
4	Example Queries and Outputs	2
5	Example Queries and Outputs	2
6	Failure Cases	3
7	Execution Control Observations	4
7.1	Base Agent (Default Temperature)	4
7.2	Configuration A: Temperature = 0.0	4
7.3	Configuration B: Temperature = 1.0	4
7.4	LoopAgent: max_iterations = 1	4
7.5	LoopAgent: max_iterations = 5	4
8	What I Would Improve Next	5
8.1	Majority Voting for Robustness	5
8.2	Strict Schema Validation at Tool Boundary	5
8.3	Strict vs. Non-Strict Inequality	5
8.4	Termination Signal for LoopAgent	5

1 Agent Architecture

The agent’s primary responsibility is twofold. First, it validates the query and determines whether it is a filtering request or an unsupported operation such as aggregation or calculation. This validation step is critical: in more complex multi-agent systems, it allows the orchestrator to route queries to the appropriate tool with higher precision, and avoid unnecessary computation.

If the query is valid, the agent parses it into a sequential chain of filter operations and calls the `filter_records` tool once per filter criterion. Each call receives the output of the previous call as `previous_results`, forming a serial pipeline. The architecture consists of three components: the LlmAgent (the reasoning engine), the `filter_records` tool, and the Runner which is an execution loop.

2 Tool Design

The `filter_records` tool accepts three arguments: `filter_type`, `filter_value`, and an optional `previous_results` list. Its first responsibility is determining the working dataset. If `previous_results` is `None`, it loads the full dataset from `sensor_data.xlsx`. Otherwise, it operates on the previously filtered records, enabling serial chaining across multiple tool calls.

The filtering logic branches on `filter_type`. For `value_range`, the tool extracts optional `min` and `max` bounds from the `filter_value` dictionary, defaulting to $-\infty$ and $+\infty$ respectively when either bound is absent. For all other filter types (`location`, `sensor_type`, `anomaly_label`), the tool performs a direct equality check against the corresponding column.

One known limitation is that the `value_range` filter does not distinguish between strict inequality ($>$) and non-strict inequality ($\geq$). Both bounds are currently treated as inclusive. To resolve this, I suggest additional arguments in the `filter_value` dictionary, such as `min_exclusive` and `max_exclusive` flags, which is left as a future improvement.

3 Example Queries and Outputs

4 Example Queries and Outputs

5 Example Queries and Outputs

The agent was tested across five categories: normal filtering queries, invalid (aggregation/calculation) queries, void-result queries, unknown feature queries, and edge cases. Overall, the agent correctly handled 12 out of 13 queries. The one partial failure was a boundary condition in the `value_range` filter, described in Section ??.

Below are two representative examples of agent responses.

Example 1: Invalid Query

Query: *“How many anomalies are in Area B?”*

Response:

I can only help with filtering questions, not with counting or aggregation.

Query	Category	Correct?	Notes
Show gas readings in Area A	Normal	Yes	4 records returned
Find vibration anomalies in Zone 2	Normal	Yes	2 records returned
Show temperature sensors with values between 20 and 30	Normal	Yes	Empty result, correctly reported
Show gas anomalies in Zone 1	Normal	Yes	1 record returned
Show all pressure readings	Normal	Yes	6 records returned
Show readings that are not anomalies in Area A	Edge case	Yes	<code>anomaly_label=False</code> handled correctly
Show gas readings above 50 that are not anomalies	Edge case	Partial	Returned <code>value=50</code> (boundary issue)
How many anomalies are in Area B?	Invalid	Yes	Correctly declined
What is the average temperature?	Invalid	Yes	Correctly declined
Which location has the most anomalies?	Invalid	Yes	Correctly declined
Show vibration readings in Zone 2 with value above 1000	Void result	Yes	Empty result, correctly reported
Show sensors with humidity above 60	Unknown feature	Yes	Correctly declined with explanation
Find readings where wind speed is below 10	Unknown feature	Yes	Correctly declined with explanation

Table 1: Test queries, categories, and agent responses.

Example 2: Normal Filtering Query

Query: “Show gas readings in Area A”

Tool calls: filter by `sensor_type=gas`, then `location=Area A`

Response:

- Asset ID: A-101, Timestamp: 2026-04-01 10:00:00, Value: 45, Anomaly: No
- Asset ID: A-113, Timestamp: 2026-04-03 08:20:00, Value: 55, Anomaly: Yes
- Asset ID: A-117, Timestamp: 2026-04-03 12:30:00, Value: 65, Anomaly: Yes
- Asset ID: A-125, Timestamp: 2026-04-04 13:30:00, Value: 35, Anomaly: No

Table 1 summarizes all tested queries and their outcomes.

6 Failure Cases

1. Boundary Condition in Value Range Filter

The `value_range` filter uses inclusive bounds ($\geq$ and $\leq$) for both `min` and `max`. As a result, the query “Show gas readings above 50 that are not anomalies” incorrectly returned a record with value exactly 50. The tool has no mechanism to distinguish between strict ($>$) and non-strict ($\geq$) inequalities without additional arguments in the `filter_value` dictionary.

2. Range format

The LLM occasionally passed `filter_value` as a raw string (e.g., "50-80") instead of the expected dictionary format `{"min": 50, "max": 80}`, causing the tool to crash with an

AttributeError. This behavior was observed across multiple configurations, suggesting it is an inherent inconsistency in how the model formats the `value_range` argument.

7 Execution Control Observations

To evaluate the effect of execution parameters, we tested five configurations on the complex multi-location query: *“Show gas anomalies in Area A and Area B with value between 50 and 80.”* Each configuration was run five times to account for stochastic variability. The ground truth answer contains three records: A-113 (Area A, value 55), A-117 (Area A, value 65), and A-103 (Area B, value 60).

Unlike local models, Gemini’s API does not expose a random seed parameter. This means the same query at the same temperature can produce different results across runs, making it difficult to evaluate configurations systematically.

7.1 Base Agent (Default Temperature)

The base `LlmAgent` with default temperature was the most reliable configuration, producing the correct answer in 4 out of 5 runs. It exhibited emergent behavior for the multi-location query — independently running two separate location filters on the full dataset and merging the results, despite no explicit instruction to do so.

7.2 Configuration A: Temperature = 0.0

At temperature 0.0, the agent was the least reliable, producing correct results in only 2 out of 5 runs. The remaining runs either returned no response or failed with a `MALFORMED_FUNCTION_CALL` error. Deterministic mode made the model too rigid to handle the ambiguity of a multi-location query.

7.3 Configuration B: Temperature = 1.0

At temperature 1.0, the agent produced correct results in 3 out of 5 runs. Failures included incorrect chaining (filtering Area B within Area A results) and occasional refusals where the agent incorrectly classified the query as requiring aggregation. Higher temperature introduced reasoning instability, and in some runs the LLM passed `filter_value` as a raw string instead of the expected dictionary, causing a tool-level error.

7.4 LoopAgent: max_iterations = 1

With a single iteration, the agent frequently returned empty results (3 out of 5 runs). The inner `LlmAgent` did not have enough steps to complete the full multi-location filter chain before being cut off.

7.5 LoopAgent: max_iterations = 5

With five iterations, accuracy improved to 4 out of 5 runs, matching the base agent. However, a significant behavioral issue emerged: after producing the correct answer on the first iteration, the agent continued looping and repeated the same answer verbatim for the remaining iterations. This indicates that `LoopAgent` lacks a termination signal — it does not know the task is complete and keeps running until `max_iterations` is exhausted which leads to wasting tokens.

8 What I Would Improve Next

8.1 Majority Voting for Robustness

The most impactful improvement would be replacing `max_iterations` with a majority voting strategy. For a deterministic filtering task, running the same query multiple times at a moderate temperature and selecting the most frequent answer is more reliable than iterating the same agent repeatedly. Unlike `LoopAgent`, majority voting introduces genuine diversity across runs rather than redundant repetition of the same reasoning chain.

8.2 Strict Schema Validation at Tool Boundary

The `filter_records` tool currently returns a soft error message when `filter_value` is not a dictionary for `value_range` queries. A more robust design would enforce strict schema validation before execution.

8.3 Strict vs. Non-Strict Inequality

The `value_range` filter treats all bounds as inclusive. Adding `min_exclusive` and `max_exclusive` boolean flags to the `filter_value` dictionary would allow the agent to correctly handle queries like “*above 50*” (strict) vs. “*50 or above*” (non-strict).

8.4 Termination Signal for LoopAgent

`LoopAgent` with `max_iterations > 1` repeatedly outputs the same answer after the task is complete. A termination signal — such as a special token or a secondary classifier that detects task completion — would allow early stopping and avoid redundant iterations.